

EL costo mayor del software son las personas que están trabajando en el mismo, el esfuerzo de las personas

El testing o la etapa de prueba se lleva entre el 30% y 50% del costo total de proyecto.

Testing: Proceso **DESTRUCTIVO** de tratar de encontrar defectos en el código, los cuales se asumen que están presentes. Buscamos encontrar el error de un código, por tanto hay que ir con una actitud negativa buscando el error

El mejor testing no es el que hace pasar un código de una, si no aquel que encuentra errores y hace que los desarrolladores deban resolverlos antes de salir a producción

El Testing **NO ASEGURA** calidad, no puede ser inyectada al final como un proceso extra, si no que debe ser una política constante durante todo el desarrollo de un sistema, y el testing no es el culpable de los errores, por el contrario, es el salvador del sistema por haberlos detectado, porque eso significa que antes de ser una falla que el cliente detecte en producción, es un defecto que se corrige en etapas de desarrollo lo que ahorra muchos costos monetarios, de confianza y temporales

Mucha gente se ponía y se pone a pesar como iba a ser para probar el producto cuando estaba casi al final de la vida del proyecto

No es necesario tener TODO el código para probar el software

Se puede ir diseñando los casos de prueba

Los casos de prueba es la identificación de situaciones concretas más los datos de seteo necesario para poder si uno o más criterios de aceptación se cumplen para un supuesto. Necesitamos datos concretos para darle valor a un escenario

Deberíamos determinar todas las pre condiciones, conjunto de situaciones VALIDAS

Artefacto de prueba más importante de la salida de la etapa de Prueba

Para desarrollar los casos de prueba unicamente necesitamos los requerimientos del software

4 niveles de prueba

Prueba unitaria: Ver que haga lo que se supone que debe hacer, validar que lo que hace lo hace bien, debemos **Validar y verificar**

Integración: Si todas las pruebas unitarias funcionan, todas juntas funcionan?. También llamados **pruebas de interfaces**

Sistema de versión: Probamos si una versión funciona

UAT: Prueba de aceptación de usuario. La debe de hacer el usuario y el usuario debe aceptar que el sistema o el componente hace lo que debería. Estas pruebas son realizadas en la etapa de despliegue y, en caso de recibir la aceptación del cliente de la prueba, y si este quisiera, este incremento puede ser puesto directamente en producción.

Nosotros cuando nombramos las pruebas de usuario, definimos los ESCENARIOS de los casos de prueba

Proceso:

- Planificación
 - Artefacto resultado: **Plan de prueba**
- Diseño
 - Artefacto resultado: **Casos de prueba**
- Ejecución
 - Cuando obtenemos la implementación del código, le pasamos los casos de prueba y obtenemos el **reporte de defectos**
- Cierre

Nosotros no necesitamos tener todo el código para empezar a planificar y diseñar los casos de pruebas, si no que a partir de tener los requerimientos estas actividades pueden ser realizadas

Conceptos importantes

Testing metodológico: Usar los casos de prueba.

vs

Testing Ad Hoc: TEsting no metodologico, no sistematico, no basado en la ejecución de los casos de prueba, me siento a probar a ver que salga

Error: No se trasladan, si no que se traen de una etapa anterior

vs

Defecto: Se traslado de una etapa anterior a una etapa más avanzada

Un error o un defecto puede provocar una falla en el sistema

Caso de prueba: Set de condiciones o variables bajo las cuales un tester determinará si el software funciona de manera correcta o no.

El concepto de caso de prueba surge por una necesidad. Cuando reportamos errores, uno usualmente lo suele hacer de una manera casual, como, me metí en el menú de compras, registre una compra de mercaderia y el programa se cerró. Es una descripción factible que podría hacer alguien como un cliente o un tester novicio, pero el problema es que

Error que no puede ser reproducido, es un error que no existe y no puede ser corregido

Por tanto, los casos de prueba surgen para determinar una secuencia de pasos específicos claros para poder reproducir el error y de esa manera, corregirlo

Objetivos de una prueba

- Validar: Proceso con la adecuación, vemos si hace lo que realmente debe de hacer
- Verificar: Ver si lo que hace, lo hace bien.

Si hacemos casos de prueba respecto al código que nos entreguen, sin conocer los requerimientos, vamos a poder validar los casos de prueba, NUNCA verificarlos.

Severidad de los defectos

- Bloqueante o invalidante
- Grave
- Leve
- Cosmético: Interacción como interfaces de usuario
- Mejora (No necesariamente un defecto pero algo que el de testing considera que debería mejorarse)

La **severidad** tiene que ver con el impacto que tiene en el uso del sistema, no lo mucho que me lleva a mi corregirlo. Es muy fácil definir lo que es cosmético y lo que es bloqueante, pero lo intermedio ya está más basado en el criterio definido anteriormente por el equipo

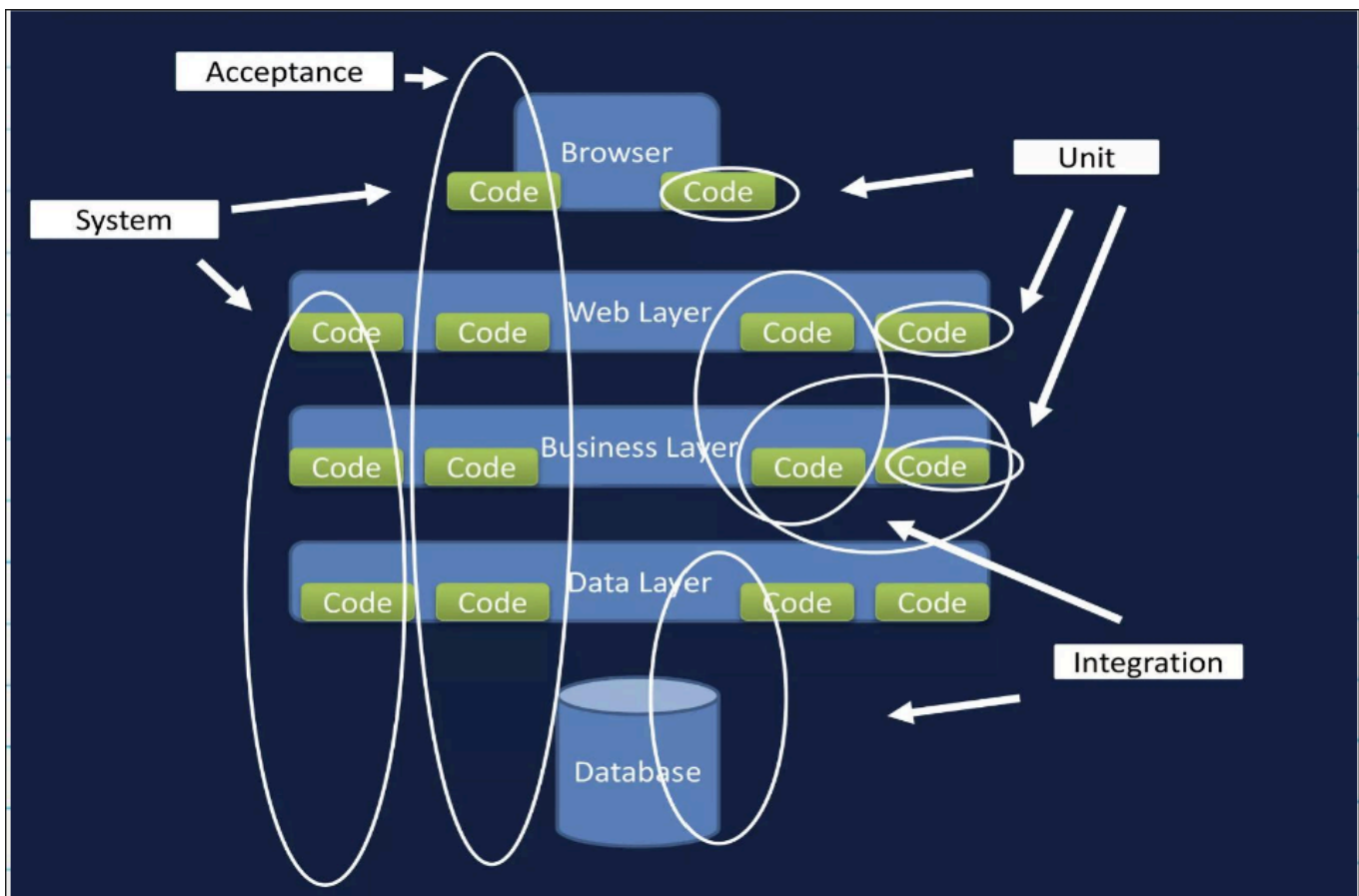
Prioridad de los bugs o defectos

- Urgente
- Alta
- Media
- Baja

La prioridad va de la mano con la severidad e indica el nivel de importancia con la que debe ser tratado el error, dado que podemos tener un defecto bloqueante o severo que no nos permite acceder a una parte de la aplicación, pero es una situación muy límite o es de una funcionalidad experimental que todavía no está implementada, y tenemos errores cosméticos de números con separación de miles, o errores de severidad baja que se detectaron como fallas del sistema en producción y que deberían ser corregidos con prioridad

Niveles de prueba

- Unitario: Prueba de funcionalidad
 - Se prueban componentes aislados de manera independiente
 - No se suele dejar constancia de los mismos,, porque se resuelven ni bien se detectan
- Integración o prueba de interfaces: 2 o más componentes que pasaron la prueba unitaria, deberían funcionar juntos
 - Deberían ser aplicadas mediante un proceso incremental, para lo cual existen 2 estrategias
 - **Top down?**
 - **Bottom-up?**
- Sistema: Probar una build entera. Se hace en el workflow de prueba
 - Determinamos si el sistema funciona en su integridad para lanzar una build
 - El ambiente de prueba debería ser lo más parecido al ambiente de producción para reducir al minimo los riesgos de incidentes debidos al ambiente
 - Probamos requerimientos funcionales y no funcionales
- Prueba de aceptación de usuario: Se hace en el workflow de despliegue por el usuario
 - Prueba realizada por **el usuario**
 - Justamente, encontrar defectos no es lo optimo en está etapa, si no que buscamos crear confianza en el cliente mostrandole un producto que funciona.



Ambientes de SW

- Desarrollo
- Prueba
- Pre-Producción
- Producción

Ciclo de prueba

Iniciamos en un ambiente de prueba, corremos todos los casos de prueba que teníamos planificados para esa versión, tengo mi reporte de defectos y termina el ciclo de pruebas. Es la ejecución de todos los casos de prueba. El primero es el ciclo 0

Deberían haber como mínimo 2 ciclos de prueba por versión que entreguemos del software.

Existen 2 estrategias

Sin regresión: Corremos todas las pruebas en el ciclo 0, una por una y fallan 3. Modificamos el código para que esas 3 resulten exitosas y en el ciclo 1, lo único que probamos son esas 3 pruebas que fallaron. Esto nos ahorra muchos costos temporales y monetarios, dado que si las otras pruebas ya pasaron, si las entradas y el resultado del código es el mismo, deberían pasar con las modificaciones hechas, no?

Con regresión: Corremos todas las pruebas en el ciclo 0, una por una y fallan 3. Modificamos el código para que esas 3 resulten exitosas y en el ciclo 1, volvemos a probar todas las pruebas, para revisar que no se hayan insertado nuevos problemas al modificar el programa. Esto nos tiene cubiertos ante fallas, pero el problema es el costo elevado que estas pruebas extras nos incurren

Con el nacimiento de los frameworks de testing automático, el hecho de probar sin regresión se volvió casi anticuado, debido a que las pruebas unitarias siempre las hacemos correr todas, incluso debemos esforzarnos para no aplicar regresión.

Tipos de pruebas

Requerimientos funcionales

- Pruebas de operación normal
- Pruebas de operación negativa
- Pruebas de proceso de negocio
- Prueba de la documentación, referido al manual de usuario y al manual de configuración

Requerimientos no funcionales

- Prueba de performance
- Prueba de Seguridad

- Prueba de Interfaces
- Entre otros

Estrategias de testing

Caja negra

- Basado en equivalencias
 - Búsqueda de clases equivalentes
 - Análisis de valores límites
- Basado en la experiencia
 - Adivinanza de defectos
 - Testing exploratorio

Caja Blanca

- Basado en el análisis de la estructura interna del software o un componente del mismo
- Cobertura de
 - Enunciados o caminos básicos
 - Sentencias
 - Decisión
 - Condición
 - Múltiple

Proceso de pruebas

Planificación y control

Verificamos que se comprende de manera correcta las necesidades del cliente y lo que se espera del programa, las partes interesadas y los riesgos inmersos en las pruebas que se van a realizar.

Se construye el plan de test, con toda la información referente a que se va a testear, como, con que costos y como se va a considerar que algo está completo, además de validarlo con el cliente.

Vemos que esta actividad es "similar" a la construcción de un documento de ERS o al conformado del product backlog con el cliente, y es que el testing debe iniciar cuando el proyecto lo hace, pensando casos de prueba, clases de equivalencia e ideando como vamos a probar el software.

Identificación y especificación

Evaluamos como vamos a testear los requerimientos, cual sería la estrategia más optima y como vamos a ir testeando para entregar el sistema, identificamos los datos necesarios, diseñamos y priorizamos los casos de prueba, así como también el entorno de pruebas.

Ejecución

Crear los casos de prueba, automatizar lo que se pueda, creación de conjuntos de pruebas para la evaluación eficiente de muchos casos, ejecutarlos, registrar los resultados y compararlos con los datos esperados

Evaluación y reporte

Evaluar los criterios de aceptación, generación del reporte de resultados para los stakeholders, recolección de información de aquellas pruebas completadas, verificar que los defectos hayan sido corregidos, analizar los entregables y evaluar como resultaron las pruebas en forma de retroalimentación